moOngy.
Minds on the move

React Avançado · Módulo 1

Sessão 1

Fundamentos do React 19 e TypeScript

ter 3 nov 2026 · 19h–21h

João Gonçalves · joaogoncalves.net

Quem sou



João Gonçalves

- Senior front-end developer
- Formador e consultor — Angular, React, JavaScript
- joaogoncalves.net
- edu@joaogoncalves.net
- WhatsApp: +351 910 346 437

Como trabalho

- Modelo mental primeiro, API depois
- Código escrito **ao vivo**, convosco
- Um projeto real do início ao fim
- Perguntas a qualquer momento

Making the complicated simple.

E vocês?

Uma volta rápida — 40 segundos cada

01

Há quanto tempo trabalham com React?

02

Que versão usam hoje? Com TypeScript?

03

Estado e dados: Redux? Context? React Query?

04

O que querem levar deste curso?

O curso: 8 módulos, 11 sessões

#	Módulo	Sessões
01	Fundamentos do React 19 e TypeScript	S1
02	Rendering, reconciliation e composição · React Compiler	S2 – S3
03	Hooks avançados e custom hooks	S4
04	Routing e data loading (React Router v7)	S5 – S6
05	Formulários e Actions · Zod	S7 – S8
06	Estado: cliente vs servidor (TanStack Query, Zustand)	S9 – S10
07	Performance e concurrent UX	S11
08	Arquitetura e Server Components	S11

Ter e qui · 19h–21h · 3 nov → 17 dez · pausa de 2 semanas entre 26 nov e 10 dez

Como trabalhamos

Teoria
o modelo mental
e as APIs

→

Código
escrito ao vivo
no Skyline

→

Resumo
os slides ficam
como referência

Repo

- `github.com/JoaoGoncalves/skyline`
- Branch `main` · uma tag por sessão
- Perdido? `git checkout sessao-NN`

Recebem

- Tudo no repo: o código e a pasta `curso/` (slides, exercícios, snippets)
- O código do Skyline, sessão a sessão
- Exercício de revisão no fim das sessões

Skyline: o projeto que vamos construir

Pesquisa e reserva de voos — uma camada por sessão

S1
Setup · tipos · componentes

S2–3
Composição · Compiler

S4
Custom hooks · fetch

S5–6
Rotas · loaders · erros

S7–8
Reserva · Actions · Zod

S9–10
TanStack Query · Zustand

S11
Transitions · arquitetura · RSC

Stack: React 19.3 · TypeScript · Vite · json-server — e, mais à frente, React Router v7, TanStack Query, Zustand e Zod

Agenda de hoje

01	**Setup moderno**	Vite · TypeScript · json-server · estrutura	≈ 25 min
02	**Componentes tipados**	Props · children · eventos · tipos do domínio	≈ 35 min
03	**React 19: o que mudou**	ref como prop · <title> · Context · o que saiu	≈ 25 min
04	**Revisão de Hooks**	Três perguntas para afinar o modelo mental	≈ 15 min

Bloco 01

Setup moderno

Vite, TypeScript e uma API local — a base de todas as sessões



A stack de 2026

Peça	Versão	Papel
React	19.3	a biblioteca de UI
TypeScript	6.0	tipos — `strict` ligado por omissão
Vite	8	dev server com HMR + build de produção
oxlint	1.x	linter (vem no template do Vite)
json-server	0.17	API REST a partir de um ficheiro JSON
Create React App	—	descontinuado em 2025

Porquê Vite

- Arranque instantâneo (ES modules nativos)
- HMR que preserva o state
- Sem servidor React: uma **SPA** pura
- O que o React recomenda para SPAs

Frameworks (Next.js, React Router framework mode) acrescentam o servidor — fica para a Sessão 11.

Anatomia de um projeto Vite + TypeScript

```
skyline/
├─ index.html          ← a entrada: carrega /src/main.tsx
├─ db.json             ← dados da API local
├─ vite.config.ts
├─ tsconfig.json       ← só referências
│  ├─ tsconfig.app.json   (src/)
│  └─ tsconfig.node.json  (vite.config.ts)
└─ src/
   ├─ main.tsx         ← createRoot + <StrictMode>
   └─ App.tsx
```

Duas flags a conhecer

- `verbatimModuleSyntax` → tipos importam-se com `import type`
- `erasableSyntaxOnly` → sem `enum` nem `namespace`: usamos **union types**
- `strict` já vem ligado (TS 6)

O TypeScript só verifica tipos (`tsc -b`). Quem transforma e serve o código é o Vite.

Uma API local com json-server

Um ficheiro JSON → uma API REST que grava de verdade

Pedido	Resultado
GET /flights	os 30 voos
GET /flights/5	o voo com id 5
GET /flights?from=Lisbon&to=Porto	filtro por igualdade (exato)
GET /flights?delay_gte=20	operadores: _gte _lte _ne _like
GET /flights?_sort=date&_order=asc	ordenação
GET /flights?_page=1&_limit=6	paginação
POST · PUT · PATCH · DELETE	gravam no db.json

```
// package.json
"scripts": {
  "dev":   "vite",
  "api":   "json-server --watch
           db.json --port 3000",
  "start": "run-p api dev"
}
```

Atenção

- Filtros **case-sensitive**: `lisbon → []`



Da teoria ao código

Agora, vamos escrever o código

- Criar o Skyline com o template react-ts do Vite
- Instalar json-server 0.17 e npm-run-all2 — e arrancar tudo com npm start
- Limpar o template e criar a estrutura de pastas

Ficheiros: package.json · db.json · src/index.css · src/App.tsx

Criar o Skyline e arrancar a API

```
npm create vite@latest skyline -- --template react-ts
cd skyline
npm install

# API local + correr tudo com um comando
npm install -D json-server@0.17 npm-run-all2

npm pkg set scripts.api="json-server --watch db.json --port 3000"
npm pkg set scripts.start="run-p api dev"

npm start
#  → http://localhost:3000/flights   (json-server)
#  → http://localhost:5173           (Vite)
```

A reter

- `json-server@0.17` — **fixar** a versão
- `run-p` corre os scripts em paralelo
- `db.json` na raiz do projeto, não em `src/`
- Testar: `/flights?from=Lisbon&to=Porto` → 5 voos

Limpar o template e organizar as pastas

```
src/
├─ main.tsx
├─ App.tsx
├─ index.css              ← CSS do Skyline (fornecido)
├─ ui/                    ← genéricos, sem domínio
│  ├─ Button.tsx
│  └─ Panel.tsx
└─ features/
   └─ flights/            ← tudo o que é de voos
      ├─ data/flight.ts
      ├─ FlightCard.tsx
      ├─ FlightList.tsx
      └─ FlightSearch.tsx
```

Regras simples

- Apagar `src/assets` e `App.css`
- `ui/` não conhece voos
- `features/flights` pode usar `ui/`, nunca o contrário
- Arquitetura a sério: Sessão 11

Bloco 02

Componentes tipados

Props, children e eventos com TypeScript — e o domínio em tipos

Tipar props

```tsx
import type { ReactNode } from 'react';

type PanelProps = {
  title: string;                  // obrigatória
  actions?: ReactNode;            // opcional
  children: ReactNode;            // o que vai lá dentro
};

export function Panel({ title, actions, children }: PanelProps) {
  // …
}

<Panel title="Flight Search" actions={<span>2 selected</span>}>
  <FlightList … />
</Panel>
```

A reter

- Props = **um** objeto, tipado na desestruturação
- Default no parâmetro: `selected = false`
- `ReactNode` para children e “slots”
- Callbacks: `onSelectedChange(selected)`
- `React.FC`: dispensável

Estender elementos nativos e tipar eventos

```tsx
import type { ComponentProps } from 'react';

type ButtonProps = ComponentProps<'button'> & {
  variant?: 'primary' | 'ghost';
};

export function Button({ variant = 'primary', type = 'button',
                         className, ...rest }: ButtonProps) {
  return <button type={type}
    className={`btn btn-${variant} ${className ?? ''}`} {...rest} />;
}
```

Evento	Tipo
onChange	ChangeEvent<HTMLInputElement>
onSubmit	SubmitEvent<HTMLFormElement>
onClick	MouseEvent<HTMLButtonElement>
FormEvent ⚠	deprecated nos tipos do React 19

`ComponentProps<'x'>` + `...rest`: o componente aceita **tudo** o que o elemento nativo aceita — sem listar nada à mão.

O domínio em tipos

```ts
export type FlightClass = 'economy' | 'business' | 'first';

export interface Price { flightClass: FlightClass; amount: number; }
export interface Aircraft { type: string; registration: string; }

export interface Flight {
  id: number;
  from: string;
  to: string;
  date: string;      // ISO
  delayed: boolean;
  delay: number;     // minutos
  aircraft: Aircraft;
  prices: Price[];
}
```

Porquê assim

- **Union de literais** em vez de `enum`: sem código gerado
- `date` fica `string` (ISO) — `Date` só para mostrar
- Os tipos **não validam** o JSON em runtime → Zod, Sessão 8



Da teoria ao código

Agora, vamos escrever o código

- Os tipos do domínio e 10 voos estáticos (temporários)
- Dois componentes genéricos: Button e Panel
- FlightCard e FlightList, com o cesto (basket) de voos selecionados

Ficheiros: features/flights/data/* · ui/Button.tsx · ui/Panel.tsx · FlightCard.tsx · FlightList.tsx

Dois componentes genéricos: Button e Panel

```tsx
// ui/Panel.tsx
type PanelProps = {
  title: string;
  actions?: ReactNode;
  children: ReactNode;
};

export function Panel({ title, actions, children }: PanelProps) {
  return (
    <section className="panel">
      <header className="panel-header">
        <h2>{title}</h2>
        {actions}
      </header>
      <div className="panel-body">{children}</div>
    </section>
  );
}
```

A reter

- `Button`: o do slide anterior, tal e qual
- `type = 'button'` por omissão: num `<form>` o default do HTML é **submit**
- `variant="danger"` → erro de compilação
- `actions`: um slot — qualquer `ReactNode`

FlightCard: o voo, o estado de seleção e um callback

```tsx
type FlightCardProps = {
  flight: Flight;
  selected?: boolean;
  onSelectedChange: (selected: boolean) => void;
};

const dateFormat = new Intl.DateTimeFormat('pt-PT', …);

export function FlightCard({ flight, selected = false,
                             onSelectedChange }: FlightCardProps) {
  const lowestPrice = Math.min(...flight.prices.map((p) => p.amount));
  return (
    <article className={selected ? 'card selected' : 'card'}>
      {/* from → to · badge de atraso · id · data */}
      <p className="price">from {lowestPrice} €</p>
      <Button variant={selected ? 'ghost' : 'primary'}
              onClick={() => onSelectedChange(!selected)}>
        {selected ? 'Remove' : 'Select'}
      </Button>
    </article>
  );
}
```

A reter

- O card **não** guarda estado
- Comunica para cima com um **callback**
- Derivados calculam-se no render
- `Intl` criado fora do componente

Onde ficámos

Parámos no FlightCard — continuamos na quinta, a partir daqui

Feito hoje

- Vite + TypeScript (`strict`)
- `json-server@0.17` + `run-p`
- Tipos do domínio + 10 voos estáticos
- `Button`, `Panel`, `FlightCard`

A reter

- Props tipadas na desestruturação
- `ReactNode` para children e slots
- `ComponentProps<'x'>` + `...rest`
- Estado no pai, **callback** para cima

Na Sessão 2

- `FlightList` e keys
- React 19: `ref` como prop, `useId`, `<title>`
- `SearchInput` + `FlightSearch`
- Revisão de Hooks · rendering

O código de hoje está no repo: `github.com/JoaoGoncalves/skyline` · tag `sessao-01`

Para quinta

O ambiente a funcionar é obrigatório — é a primeira coisa que vamos verificar

1 · Ambiente (obrigatório)

- `node -v` → **22.12+** (ou 24 LTS)
- `git clone https://github.com/JoaoGoncalves/skyline.git`
- `git checkout sessao-01` · `npm install`
- `npm start` → **duas** saídas: `:3000` (API) e `:5173` (Vite)

2 · Exercício de revisão (≈ 20 min)

- `ui/Badge`: `ComponentProps<'span'>` + `variant: 'info' | 'warn'`
- Usar no `FlightCard` (atraso / on time)
- Dois `FlightCard` temporários no `App` para ver o resultado
- Extra: `lowestPrice(flight)` em `data/price.ts`

Enunciado: `curso/exercicios/sessao-01.md` · se o setup falhar, enviem o erro para edu@joaogoncalves.net antes de quinta

moOngy.
Minds on the move

Obrigado · até quinta, 5 nov

edu@joaogoncalves.net · WhatsApp +351 910 346 437

`github.com/JoaoGoncalves/skyline · tag sessao-01`